Checklist - kacchiOS Lab Project (CSE 3202)

Every requirement of the original assignment is listed with its original category and weight. Status: 'Implemented and tested' means the feature runs in QEMU and was verified with the test described in this document.

Must Include (7 x 10 = 70%)

#	Requirement	Status	Where in the code	Test and result
1	Memory Manager - Stack, Heap allocation	Implemented and tested	memory.c: allocate_stack(), allocate_heap()	Shell command 'memtest': allocates stacks and heap blocks, checks addresses and sizes. PASS
2	Process Manager - Process table	Implemented and tested	process.c: process_table[16], process_get()	Shell command 'ps' prints the whole table; empty slots are visible. PASS
3	Process Manager - Process creation	Implemented and tested	process.c: process_create()	'spawn' / 'server' create processes; the new pid runs and prints heartbeats. PASS
4	Process Manager - State transition	Implemented and tested	process.c: process_set_state() (single transition point)	'ps' shows READY/CURRENT/BLOCKED/TERMINATED at the right moments; illegal transitions are refused. PASS
5	Process Manager - Process termination	Implemented and tested	process.c: process_exit(), process_terminate(); scheduler.c: cleanup_terminated()	'kill <pid>'; the slot and stack are freed and reused by the next 'spawn'. PASS
6	Scheduler - Clear policy to schedule	Implemented and tested	scheduler.c: pick_next_ready() (round robin + aging, documented in scheduler.h)	Interleaved worker heartbeats between shell prompts; all 15 workers make progress. PASS
7	Scheduler - Context switch	Implemented and tested	isr.S: timer_isr (pusha / ESP swap / popa / iret); scheduler.c: scheduler_tick()	Workers preempt the shell and the shell resumes exactly where it stopped (typing continues while heartbeats appear). Real ESP swap, verified in QEMU. PASS

Good to have (4 x 5 = 20%)

#	Requirement	Status	Where in the code	Test and result
1	Memory Manager - Stack deallocation	Implemented and tested	memory.c: free_stack() (with double free / invalid pointer checks)	'kill' frees the stack; 'mem' shows the slot count restored; next 'spawn' reuses it. PASS
2	Memory Manager - Heap deallocation	Implemented and tested	memory.c: free_heap() (coalesces neighbours)	'memtest': free bytes return to the exact initial value after freeing all blocks. PASS
3	Process Manager - Utility functions to get process specific functions	Implemented and tested	process.c: process_current_pid(), process_get_state(), process_get_name(), process_get_stack_size(), process_count()	'ps' is built entirely from these functions. PASS
4	Scheduler - Configurable time quantum	Implemented and tested	scheduler.c: scheduler_set_quantum() / scheduler_get_quantum(); shell command 'quantum'	'quantum 1', 'quantum 50', 'quantum 100' verified; values are clamped to 1..100. PASS

Bonus (4 x 2.5 = 10%)

#	Requirement	Status	Where in the code	Test and result
1	Memory Manager - Optimized memory allocation	Implemented and tested	memory.c: first fit + block splitting on allocate, coalescing on free	'memtest': a 50-byte request reuses a freed 200-byte block; after freeing everything the heap is one single free block again. Limitation: no best-fit/buddy system. PASS
2	Process Manager - Add more states	Implemented and tested	process.h/.c: PROCESS_BLOCKED + blocked->READY wake-up in ipc_send()	'server' shows BLOCKED in 'ps'; a message wakes it; killing a BLOCKED process works. PASS
3	Process Manager - Inter-process communication (IPC)	Implemented and tested	process.c: ipc_send(), ipc_receive(), ipc_receive_wait() (one-slot mailbox)	'send 1 hello' -> worker prints it; 'send 0 x' + 'inbox' round trip; full mailbox and bad pid are rejected. PASS

#	Requirement	Status	Where in the code	Test and result
4	Scheduler - Implement Aging	Implemented and tested	scheduler.c: wait_ticks / SCHED_AGING_STEP inside pick_next_ready()	'busy 300': the higher-priority shell spins, yet lower-priority workers still get the CPU (their heartbeats appear during the busy loop). PASS

Test environment and honest status notes

- All tests were executed in QEMU 11.0 (qemu-system-i386, -m 64M) with the kernel built by 'make' (gcc -m32, freestanding).
- Every 'PASS' above comes from actually running the shown shell commands and observing the printed output, not from code inspection alone.
- A 30 second soak test (repeated create/terminate cycles, quantum changes, memtest under load) completed without crashes or memory corruption.
- Not implemented by design: user mode, paging, multiple CPUs, per-exception handlers (see Project_Report.pdf, Limitations).